

GitHub Actions → AWS EC2

Complete Classroom CI/CD Demo Guide — Copy/Paste Edition

All code blocks are formatted for direct copying into student files.

Goal: Push a simple HTML/CSS application to GitHub and automatically deploy it to Ubuntu AWS EC2 using GitHub Actions, SSH and Nginx.

Developer

■ git push



GitHub Repository

■ push trigger



GitHub Actions

■■■ Checkout

■■■ Test

■■■ Configure SSH

■■■ Deploy

■■■ Verify

■ SSH



AWS EC2 → Nginx → Web Application

1. Learning objectives

Students should be able to:

- Explain Git vs GitHub vs GitHub Actions.
- Create a Git repository and push source code to GitHub.
- Create and understand a GitHub Actions workflow.
- Understand triggers, jobs, steps and runners.
- Use GitHub Secrets for deployment credentials.
- Deploy a web application from GitHub Actions to AWS EC2.
- Demonstrate continuous deployment by pushing a new application version.
- Troubleshoot workflow-trigger and SSH/network failures.

Key terminology

Term	Meaning
Git	Distributed version-control system.
GitHub	Platform for hosting Git repositories and collaboration.
GitHub Actions	GitHub automation and CI/CD capability.
Runner	Machine that executes GitHub Actions jobs.
EC2	AWS virtual machine used as deployment target.
Nginx	Web server serving deployed files.
GitHub Secret	Repository value kept outside source code for sensitive data.

2. AWS EC2 setup

Launch Ubuntu Server 24.04 LTS. Use a small classroom instance and create/select an EC2 key pair.

Rule	Port	Source	Purpose
HTTP	80	0.0.0.0/0	View the website
SSH	22	Temporary lab access	GitHub-hosted runner connection

For this classroom implementation, SSH is temporarily opened to 0.0.0.0/0 because the GitHub-hosted runner does not have the trainer's fixed IP. Tighten/remove this rule after the lab.

Connect to EC2 and install Nginx

```
ssh -i <your-key.pem> ubuntu@<EC2-PUBLIC-IP>
```

```
sudo apt update
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl start nginx
sudo systemctl status nginx
```

Verify the default Nginx page at:

```
http://<EC2-PUBLIC-IP>
```

3. Create the application

Create a local folder named devops-github-demo.

index.html — copy exactly

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DevOps CI/CD Demo</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>DevOps CI/CD Demo</h1>
    <p>
      This application is deployed using
      <strong>GitHub Actions</strong>
      to an
      <strong>AWS EC2</strong> instance.
    </p>
    <div class="info">
      <p><strong>Source:</strong> GitHub</p>
      <p><strong>CI/CD:</strong> GitHub Actions</p>
      <p><strong>Server:</strong> AWS EC2</p>
      <p><strong>Web Server:</strong> Nginx</p>
    </div>
  </div>
</body>
</html>
```

style.css — copy exactly

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f4f6f8;
}

.container {
  width: 70%;
  margin: 100px auto;
  padding: 40px;
```

```
    background: white;
    text-align: center;
    border-radius: 10px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.1);
}

h1 {
    margin-bottom: 20px;
}

.info {
    margin-top: 30px;
    padding: 20px;
    background: #f0f3f6;
    border-radius: 8px;
}
```

4. Initialize Git and push to GitHub

```
git --version
git init
git status
git add .
git commit -m "Initial application"
git branch -M main
```

Create a GitHub repository named devops-github-demo. Do not initialize it with another README, .gitignore or license.

```
git remote add origin <YOUR-GITHUB-REPOSITORY-URL>
git remote -v
git push -u origin main
```

Verify index.html and style.css appear in the GitHub repository.

5. Prepare EC2 for deployment

```
sudo chown -R ubuntu:ubuntu /var/www/html
ls -ld /var/www/html
```

In GitHub: Repository → Settings → Secrets and variables → Actions → New repository secret.

Secret	Value
EC2_SSH_KEY	Complete contents of the EC2 .pem private key
EC2_HOST	EC2 public IPv4 address only
EC2_USER	ubuntu

Never put the private key in the repository, YAML, README, screenshots or chat.

6. Create .github/workflows/deploy.yml

```
name: Deploy to AWS EC2
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
jobs:
```

```
  deploy:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout source code
```

```
        uses: actions/checkout@v4
```

```
      - name: Test application
```

```
        run: |
```

```
          test -f index.html
```

```
          test -f style.css
```

```
          echo "Application files found successfully"
```

```
      - name: Configure SSH
```

```
        run: |
```

```
          mkdir -p ~/.ssh
```

```
          echo "${{ secrets.EC2_SSH_KEY }}" > ~/.ssh/ec2_key
```

```
          chmod 600 ~/.ssh/ec2_key
```

```
          ssh-keyscan -H "${{ secrets.EC2_HOST }}" >> ~/.ssh/known_hosts
```

```
      - name: Deploy application
```

```
        run: |
```

```
          scp -i ~/.ssh/ec2_key index.html style.css "${{ secrets.EC2_USER }}"@${{ secrets.EC2_HOST }}:/var/www/html/
```

```
      - name: Verify deployment
```

```
        run: |
```

```
          ssh -i ~/.ssh/ec2_key "${{ secrets.EC2_USER }}"@${{ secrets.EC2_HOST }} "ls -l /var/www/html/"
```

Important: preserve the indentation exactly. YAML uses indentation to define structure. Use spaces, not tabs.

What the workflow means

Section	Purpose
on → push → main	Starts the workflow after a push to main.
jobs → deploy	Defines the deployment job.
runs-on	Uses a GitHub-hosted Ubuntu runner.
checkout	Gets repository code onto the runner.
Test	Checks required application files.
Configure SSH	Creates the private key file and known_hosts entry.
scp	Copies application files to EC2.
ssh verify	Connects to EC2 and lists deployed files.

7. Commit and run the pipeline

```
git add .github/workflows/deploy.yml
git commit -m "Add GitHub Actions deployment workflow"
git push
```

Open GitHub → Actions → Deploy to AWS EC2.

```
Set up job
↓
Checkout source code
↓
Test application
↓
Configure SSH
↓
Deploy application
↓
Verify deployment
↓
Complete job
```

After a successful run, open <http://>. The custom page should replace the default Nginx page.

8. Successful workflow — example from the trainer demo

← Deploy to AWS EC2

✓ Add GitHub Actions deployment workflow #2

Re-run all jobs

🕒 Latest #2 ▾

⋮

🏠 Summary

All jobs

✓ deploy

Run details

🕒 Usage

📄 Workflow file

> Annotations

1 warning and 1 notice

deploy

succeeded now in 6s

🔍 Search logs

🔄 ⚙️

> ✓ Set up job

1s

> ✓ Checkout source code

0s

> ✓ Test application

0s

> ✓ Configure SSH

1s

> ✓ Deploy application

1s

> ✓ Verify deployment

1s

> ✓ Post Checkout source code

0s

> ✓ Complete job

0s

Every deployment step completed successfully.

9. Troubleshooting

Issue A — “No event triggers defined in on”

Make sure the trigger is exactly:

```
on:  
  push:  
    branches:  
      - main
```

Use spaces, not tabs, and ensure the branch is main.

Issue B — Configure SSH fails at ssh-keyscan

If ssh-keyscan fails, the GitHub-hosted runner cannot reach EC2 on TCP 22. In our build, the EC2 security group allowed SSH only from the trainer's IP, while the GitHub runner came from another public IP.

Temporary classroom fix: allow TCP 22 from 0.0.0.0/0, rerun the job, then remove/restrict the rule after the demonstration.

10. Demonstrate continuous deployment

Change the heading in index.html:

```
<h1>DevOps CI/CD Demo</h1>
```

to:

```
<h1>Version 2 - Automatically Deployed!</h1>
```

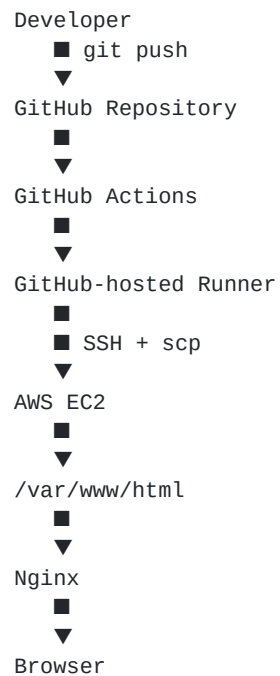
Then push the change:

```
git add .  
git commit -m "Update application to version 2"  
git push
```

The push automatically starts the workflow again. After it succeeds, refresh the EC2 URL and verify the new heading.

Teaching message: "I changed the source code, committed it and pushed it. I did not manually log in to the server to deploy the change. The pipeline performed the deployment."

11. Final architecture



Concept	What students should remember
Trigger	Push to main starts the workflow.
CI	Automated checkout and validation.
CD	Automated deployment to EC2.
Runner	Executes the workflow; it is not the EC2 server.
Secrets	Sensitive values stay outside source code.
Security group	Controls network access to EC2.
Nginx	Serves deployed files.

12. Production considerations

- Do not permanently expose SSH TCP 22 to 0.0.0.0/0.
- Use least-privilege access and short-lived credentials where practical.
- Consider AWS Systems Manager and IAM-based access for production-oriented deployments.
- Use environment separation and stronger deployment controls.
- Use HTTPS for real applications.
- Use proper automated tests rather than only file-existence checks.
- Review and maintain third-party GitHub Actions used by workflows.

After the classroom demo, remove the temporary public SSH rule or restore a restricted source range.

13. Student recreation checklist

Done	Task
■	Launch Ubuntu EC2
■	Allow HTTP 80
■	Prepare SSH access for the lab
■	Install and verify Nginx
■	Create HTML/CSS application
■	Initialize Git and commit
■	Create GitHub repository
■	Push source code
■	Create EC2_SSH_KEY secret
■	Create EC2_HOST secret
■	Create EC2_USER secret
■	Create deploy.yml
■	Push workflow
■	Verify successful Actions run
■	Verify website
■	Change application and push again
■	Observe automatic redeployment
■	Restrict SSH / clean up

Final result: students can recreate the GitHub Actions → AWS EC2 CI/CD pipeline in their own accounts.